

Table of Contents

[Home](#)

[Manual](#)

[Variables](#)

[Getting Started](#)

[Overview](#)

[Localization](#)

[Getting Started](#)

[Overview](#)

[Extending Localization](#)

Manual Home of No Such Studio

Here are the list of Unity assets developed by us. Click on the asset you are interested in to go to its manual section.

- [No Such Studio: Variables](#): This asset includes utility classes to declare and use global variables in Unity.
- [No Such Studio: Localization](#): The localization asset comes in the two flavors:
 - Pro: Includes advanced features and *Variables* asset.
 - Lite: Includes the libraries for localizing Unity projects only. Free forever!

Scripting API

The code is split into 4 namespaces:

- [NoSuchStudio.Common](#) This namespace includes the shared code between all assets.
- [NoSuchStudio.Variables](#) This namespace contains the code for the *Variables* asset.
- [NoSuchStudio.Localization](#) This namespace contains the code for the *Localization* asset.
- [NoSuchStudio.DataStorage](#) This namespace is still under development and will contains additional classes for saving data and retrieving them. This namespace is included with both the *Variables* and *Localization* assets.

No Such Studio's *Variables* Asset

No Such Studio: Variables is a beginner-friendly package for Unity. It allows you to create global variables that are accessible from any script.

- No more looking for `GameObject`s and their `Component`s to look for shared variables.
- A simple observable pattern to receive callbacks when variables change.

❑ Why No Such Studio: *Variables*?

❑ Set and Get Global Variables from Any Script

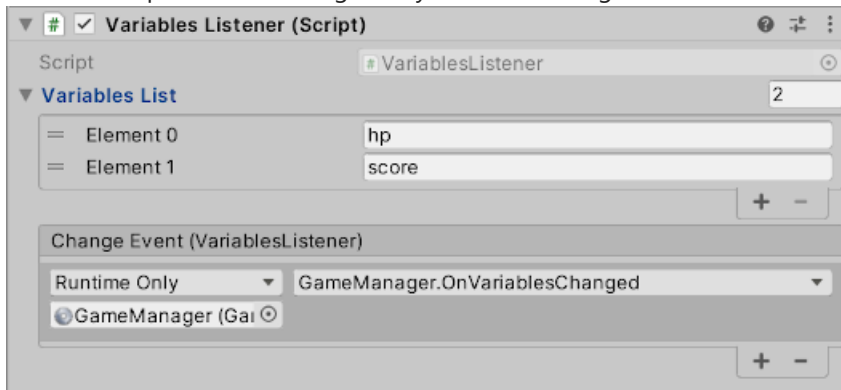
This asset makes it easy to create global variables from Unity Editor UI. You can set and get variables using the following two static methods:

```
// get player score
var playerScore = VariablesService.GetVariables("player_score");

// set player score
VariablesService.SetVariable("player_score", "10");
```

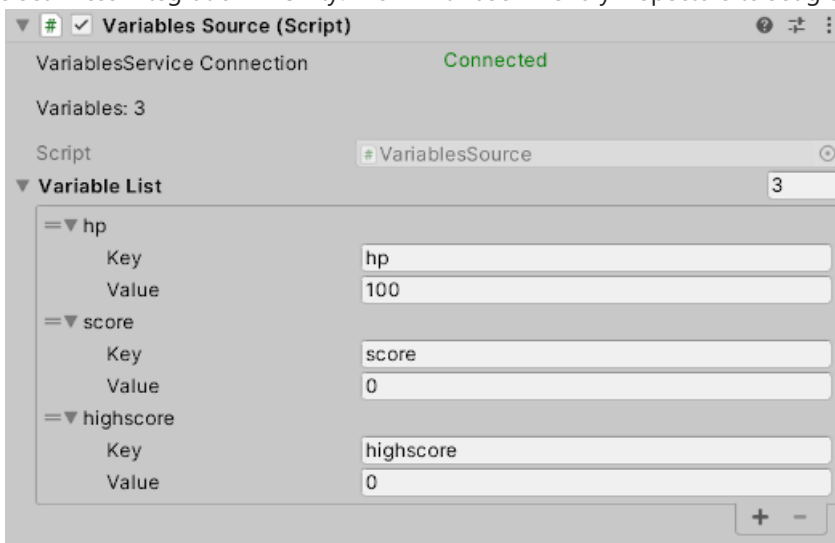
❑ Listen for Changes to Variables

You can setup callbacks through Unity Editor or through code to receive callbacks when variables change.

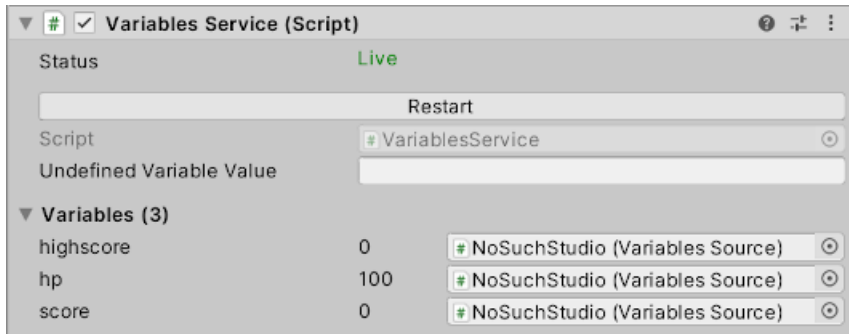


❑ Seamless Unity Editor Integration

This asset has seamless integration in Unity. Work with user friendly inspectors to set global variables and the callbacks when



they change.



☐ Tutorials, Demo Scenes and Good Documentation

This package contains demo scenes and reusable prefabs to get you started. You can always reach us at our [discord server](#) and [support forum](#) for this asset.

Geting Started

You can get the *Variables* asset from [Unity Asset Store](#).

To start using the asset, visit the [Getting Started](#) page.

Getting Started

Variables is a Unity asset that let's you create and use global variables that are accessible from anywhere in Unity. This guide will walk you through how to add the *Variables* asset to a Unity project.

1. Create an Empty Project

Create a new Unity project. It doesn't matter what type of project you create. This asset consists of scripts mainly and is compatible with any render pipeline.

2. Import the *Variables* asset into a the Project

1. Get the asset from [Unity Asset Store](#).
2. Import *Variables* into your Unity project.

Unity 2020 and Newer

In recent versions of the Unity Editor, the *Asset Store* window is gone. You need to import assets from the *package manager* window.

1. Open the *package manager* window from the *window -> package manager* menu.
2. Find No Such Studio: Variables and click Install, then Import.

Unity 2019 and older

In older versions of Unity Editor, you can import the asset through the *Asset Store* window.

1. Open the *Asset Store* window either by pressing ctrl+9 or from the *window -> asset store* menu.
2. Find No Such Studio: Variables and click Install, then Import.

This step adds the *Variables* asset under **Assets/NoSuchStudio** folder. You are all set to use the asset in your project!

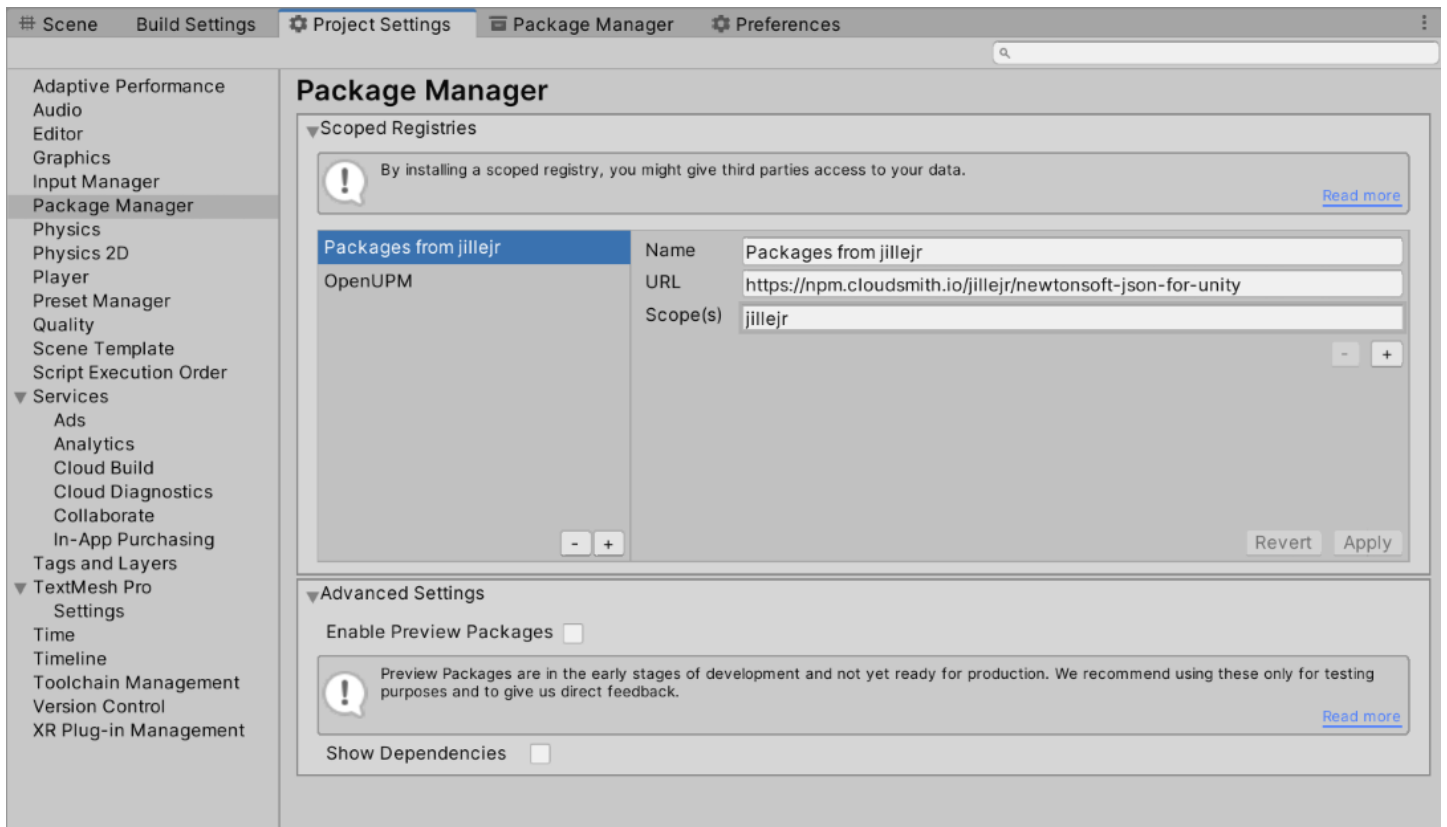
3. JSON Support for Unity 2019 and older

This step is optional. By adding these additional packages, you get to use more features *No Such Studio: Variables* as explained here.

These packages are hosted on 3rd party repositories. You need to add these repositories to Unity's UPM:

1. Open the *Project Settings* window
2. Go to the *Package Manager* tab
3. Add these two repositories.

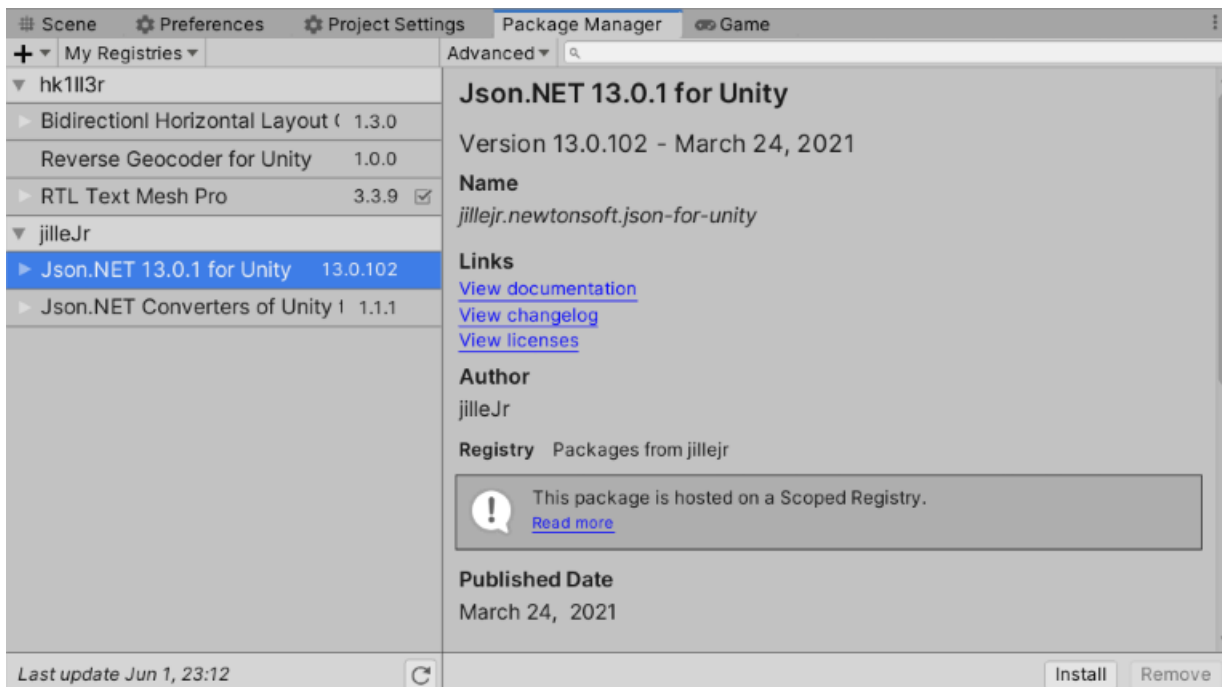
REPOSITORY NAME	URL	SCOPE
Packages from jillejr	https://npm.cloudsmith.io/jillejr/newtonsoft-json-for-unity	jillejr



No Such Studio: Variables uses *NewtonSoft JSON* package when it is present. The following features depend on it:

- Typed variables

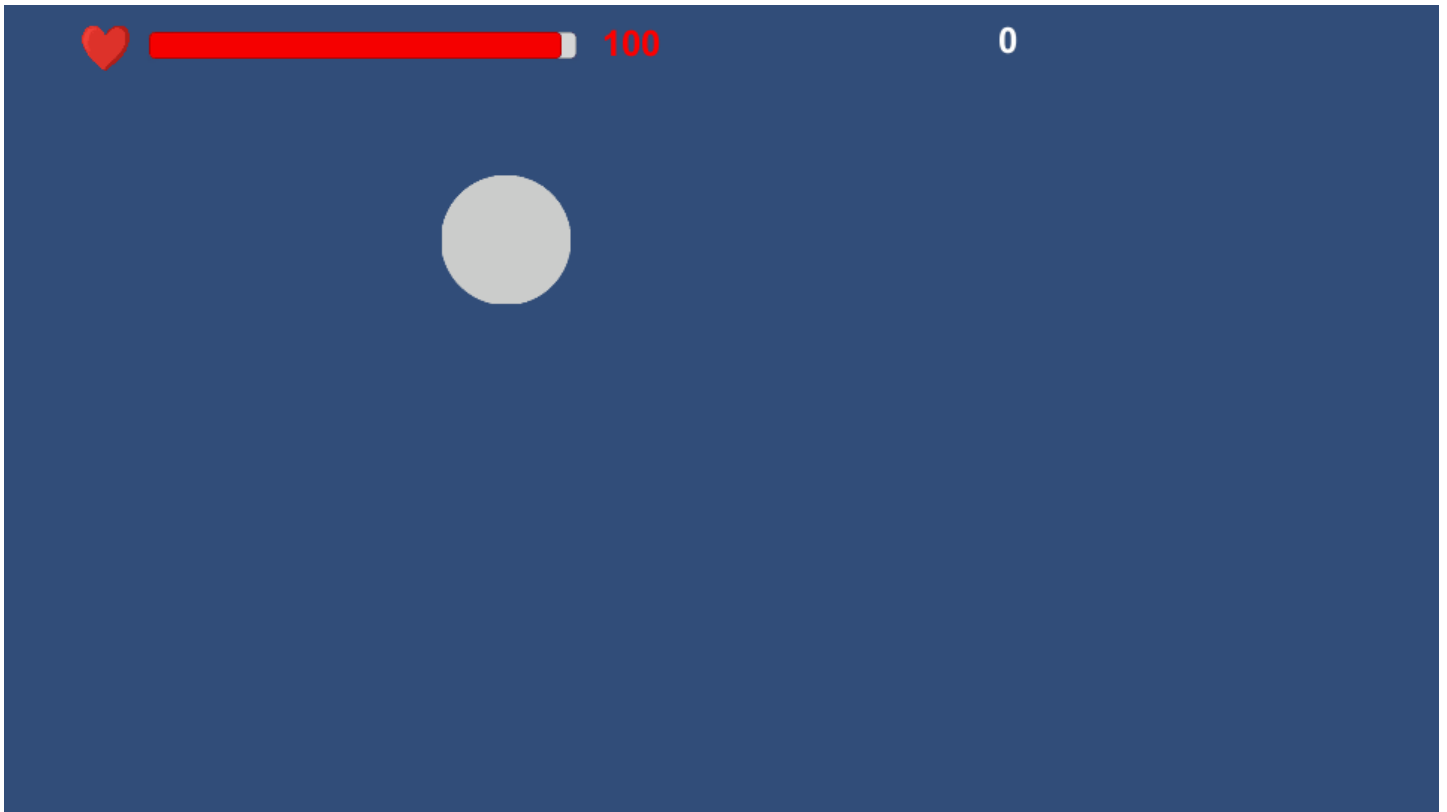
In Unity 2020 and newer the package is included by default. In 2019 and older you can add the following package to use these features.



4. Exploring the Demo Scenes

The demo scenes are a great resource for learning how to use this asset from examples.

Load the first demo scene from **Assets/NoSuchStudio/Variables/Examples/Scenes/BubbleBurstGame** by double clicking it.



Run the scene to see the game in action! The bubbles pop at random locations on the screen and the player has to click them quickly. Each bubble that is not burst in time reduces the player's health. The game ends when the player's health is 0.

Let's look at the important components of this scene:

1. `VariablesService` component (on `NoSuchStudio` `GameObject`) This component is the central part of the *Variables* system. It keeps track of all the Variables that are present in the scene and monitors their changes. Each scene that uses the Variables system should contain a single instance of this components. In a multi-scene game it makes sense to set the `GameObject` that has the `VariablesService` component (`DontDestroyOnLoad`);
2. `VariablesSource` component (on `NoSuchStudio` `GameObject`) The pre-made `VariablesSource` component is a simple store for declaring variables. You can add / remove and edit variables from the Inspector window for this component. For this simple scene, there are 3 variables defined: "hp", "score" and "highscore". Note how by modifying the variables and their values here, they also change on the `VariablesService` component.
3. `VariablesListener` component (on `GameManager` `GameObject`) This component monitors certain variables for changes and invokes a callback whenever *any* of those variables change. This way you can decouple the logic that changes a value from the logic for reacting to that change. For example in this game the `Enemy.cs` script adds to the user score whenever a bubble is clicked. The HUD UI updates the user score in a callback from that change.
4. `GameManager` and `Enemy` components These two components implement the game logic. They interact with the `Variables` system through static calls to `@"VariablesService.SetVariable"` and `@"VariablesService.GetVariable"`.

See Also

- To get better understanding of the asset and how to use it, take a look at the [Overview](#) page.
- You can access the scripting API reference [here](#).

Overview

Basics

The `Variables` system consists of three categories of components. Note that any script can set and get variable values through static methods [SetVariable](#) and [GetVariable](#) without any special implementation. The following components however need to implement [IVariablesServiceComponent](#) to be part of the variables system.

1) VariablesService

`@NoSuchStudio.Localization.VariablesService` acts as the central hub for the variables framework. There must be a single instance of this component in the scene. I recommend marking the `GameObject` [DontDestroyOnLoad](#). If there are multiple instances of this script, the one that gets enabled last will act as the registered and active instance.

Other components that participate in the process register with this service instance and are able to make new variables, watch the current variables and change their values.

2) Variable Sources

Variable sources are components that provide variables to the system. They could store the variable values on disk, in the cloud or in memory. It really depends on the component how to store their variables. This asset comes with [in-memory](#) and [persistant local storage](#) sources. You can write your own sources too. For example you could store and retrieve variables from a Google sheet or a SQL database.

3) Consumers

Components that want to listen to variable change events. The asset comes with the [VariablesListener](#) component. This flexible component can be configured from Unity Editor to listen for changes to the selected variables and invoke a callback when any variable changes.

See Also

- Read the [Getting Started](#) page to start using *No Such Studio's Variables* asset.
- You can access the scripting API reference [here](#).

No Such Localization

No Such Localization is a beginner-friendly yet feature rich localization package for Unity. I started writing the asset as part of my own games as a bilingual indie developer. Later on I factored the code into this asset so that I could share it with the game dev community. I take a good chunk out of my time to maintain and develop **No Such Localization** but I find it rewarding to do so. I've published a **Lite** version which is free and has all the essential features to localize your game. The **Pro** version has power features and is intended for larger teams. If you use **No Such Localization**, please leave me a review on Unity Asset Store. If you use the asset and like it, please support me by purchasing the asset.

Getting Started

You can get **No Such Localization** from Unity Asset store.

[Lite version](#)

[Pro version](#)

To learn how to use the asset, visit the [Getting Started](#) page.

Why **No Such Localization**

☐ **Complete Localization System for Unity**

No Such Localization is designed with simplicity in mind. Use regular components to localize your game inside Unity Editor. Your changes take effect in Edit mode like the rest of Unity UI. No need to enter play mode to see the result of language changes.

☐ **No Coding Required**

No Such Localization comes with a base collection of localizer components for images, audio sources, text mesh pro and more. You can start localizing your game by adding these components to your GameObjects.

☐ **Extensible Architecture**

With No Such Localization's extensible architecture, you can write your own localized components quickly. Learn more by visiting the [Extend No Such Localization](#) page. (beginner C# programming required)

☐ **Seamless Unity UI and Editor Integration**

No Such Localization has seamless integration in Unity. Work with pre-made localizer components, add the appropriate localizers from the context menu of other components. Never leave the environment you are familiar with.

☐ **Full Right-to-Left Language Support [Pro]**

No Such Localization has first class support for RTL languages (Arabic, Persian, Hebrew, etc.). It supports RTL layout of UI elements and automatically fixes Arabic characters.

☐ **Automatic Variable Replacement [Pro]**

No Such Localization lets you include {variables} in you translations. At runtime (and at edit time) it automatically replaces the variables with their current values.

This feature is very powerful and can be used outside the context of localization. Do you show the user score, hp, ammo, etc. in UI? Use the Variable Service from this package to update your UI when these values change.

☐ **Support for JSON and CSV files [Pro]**

JSON and CSV files are supported and support for XML files is on the way. With No Such Localization's extensible architecture, you could add support for new file types on your own.

This can help with larger projects where multiple developers and translator have to work together. Translators can work independently and create the translation files outside of Unity.

❑ **Tutorials, Demo Scenes and Good Documentation**

This package contains demo scenes and reusable prefabs to get you started. In addition I have made tutorial videos and articles to get you started as quickly as possible. Links at the top of the page.

Lite vs Pro Version

No Such Localization comes in two versions **Lite** and **Pro**.

The **Lite** version has all the essential features to localize a Unity project. It is suitable for small projects. Intended to be used by small studios, hobbyists and indie developers. You could also use the **Lite** version to try out the asset before buying.

The **Pro** version has power features that are useful for larger projects with multiple developers and translator working together. You can buy the **Pro** version to also support the developer.

FEATURE	LITE	PRO
Common Localizers for Sprite, Text, Audio Clips and more	*	*
Language Detection and Saving	*	*
Easy Setup and Debugging in Unity Editor	*	*
Great Support and Learning Material	*	*
Demo Scenes	*	*
Full RTL Support	-	*
File Translation Sources (CSV, JSON, XML)	-	*
Variable Substitution	-	*

Getting Started

No Such Studio: Localization is a Unity asset that let's you localize your game for different languages. It is easy to setup, use and extend. There are localizers included for Sprites, Text, Audio Clips and you can easily extend the asset by writing new `component localizers` and `translation sources`. This guide will get you started on the asset. I have also created a series of tutorials on youtube to help ramp you up. You can check out the videos [here](#).

In this guide we are going to add *No Such Studio: Localization* to a new Unity project and explore the Demo scenes that are included with the asset. The demo scenes are a great resource for learning from examples.

1. Create an Empty Project

Create a new Unity project. It doesn't matter what type of project you create. This asset consists of scripts mainly and is compatible with any render pipeline.

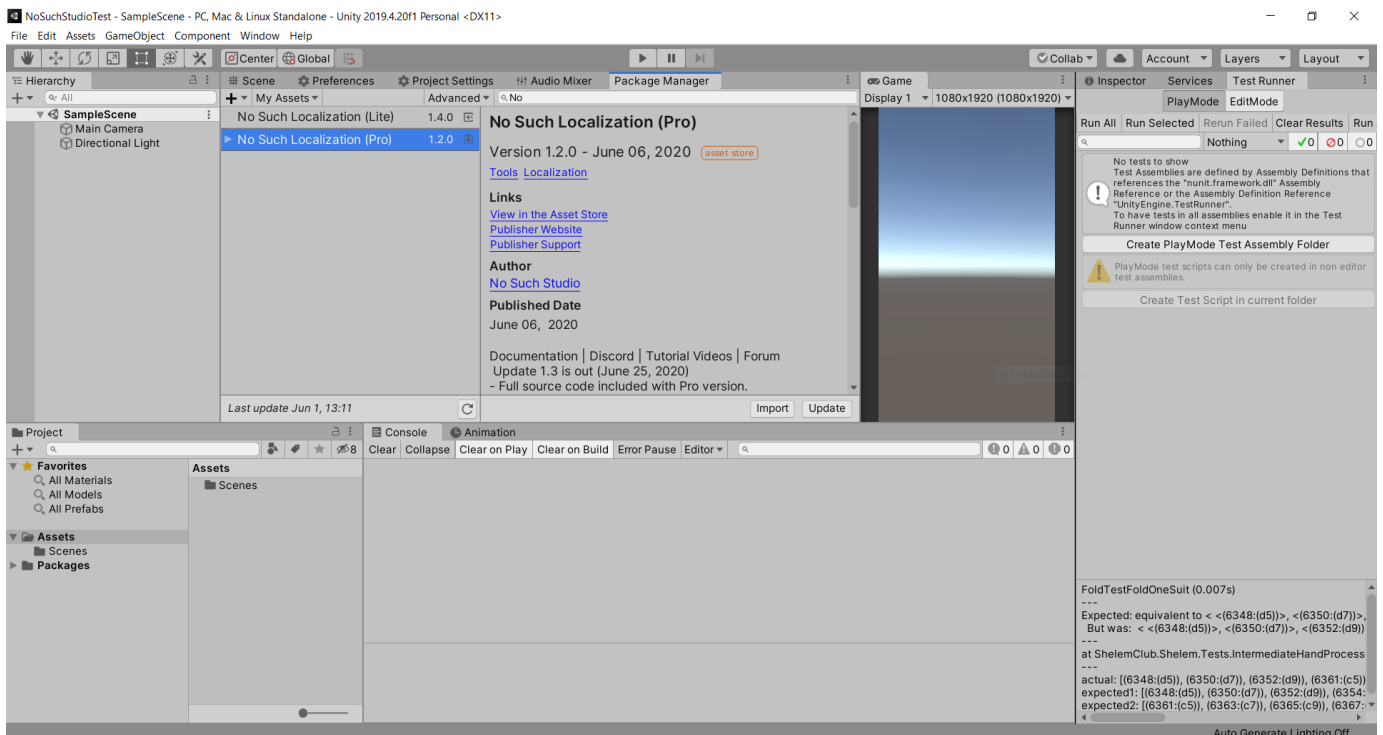
2. Add the *Localization* asset to the Project

You can download and import the asset like any other asset from the Unity asset store.

Unity 2020 and newer

In recent versions of the Unity Editor, the *Asset Store* window is gone. You need to import assets from the *package manager* window.

1. Open the *package manager* window from the *window* -> *package manager* menu.
2. Find *No Such Studio: Localization* and click Install, then Import.



Unity 2019 and older

In older versions of Unity Editor, you can import the asset through the *Asset Store* window.

1. In Editor open the asset store window. (shortcut ctrl + 9)
2. Search for "No Such Studio: Localization". There is a Lite and a Pro version. Check out the [home page](#) to learn more about the differences between the two version.

3. Press "Import", then "Ok".

This step adds the *No Such Studio: Localization* asset under **Assets/NoSuchStudio** folder. You are all set to use the asset in your project!

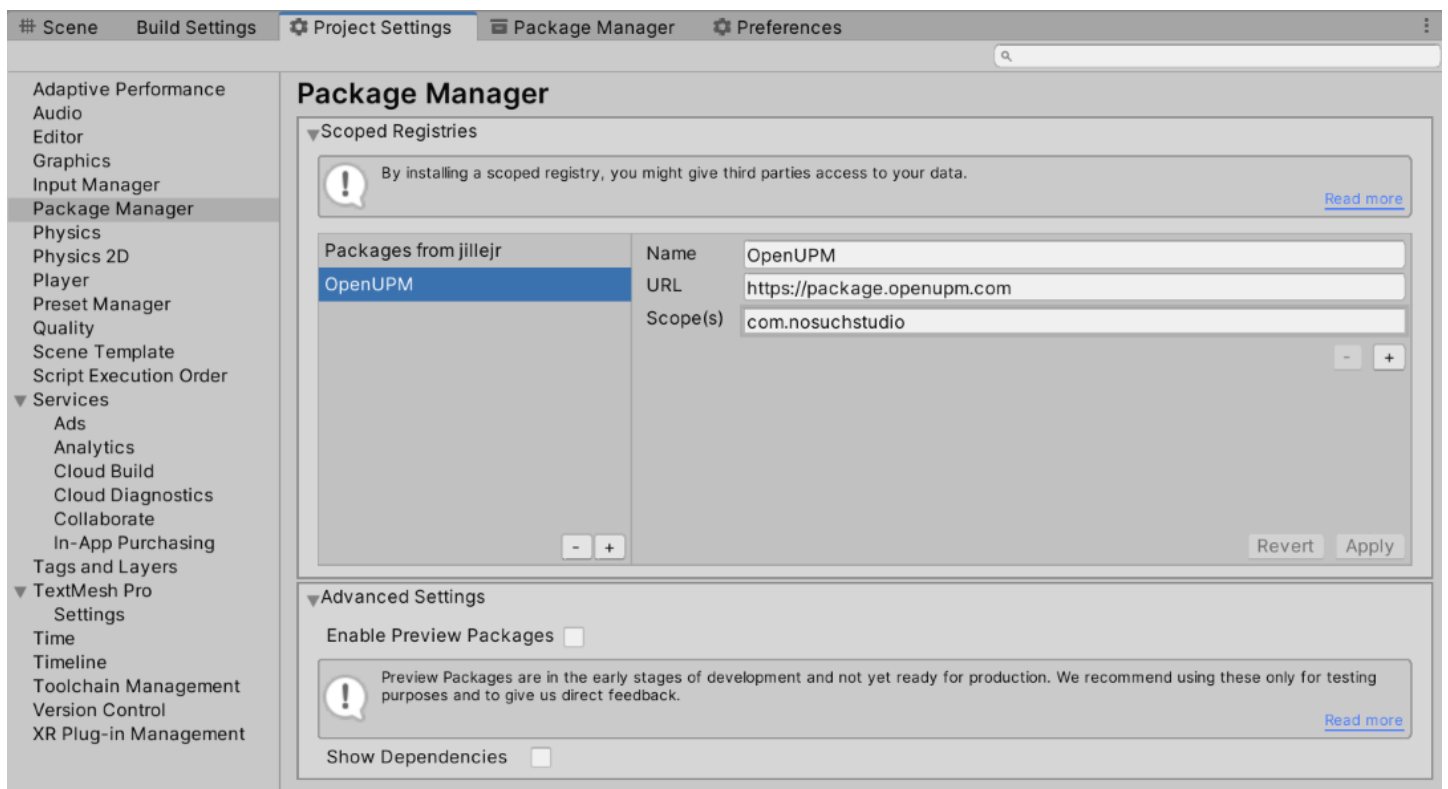
3. Add Additional Packages

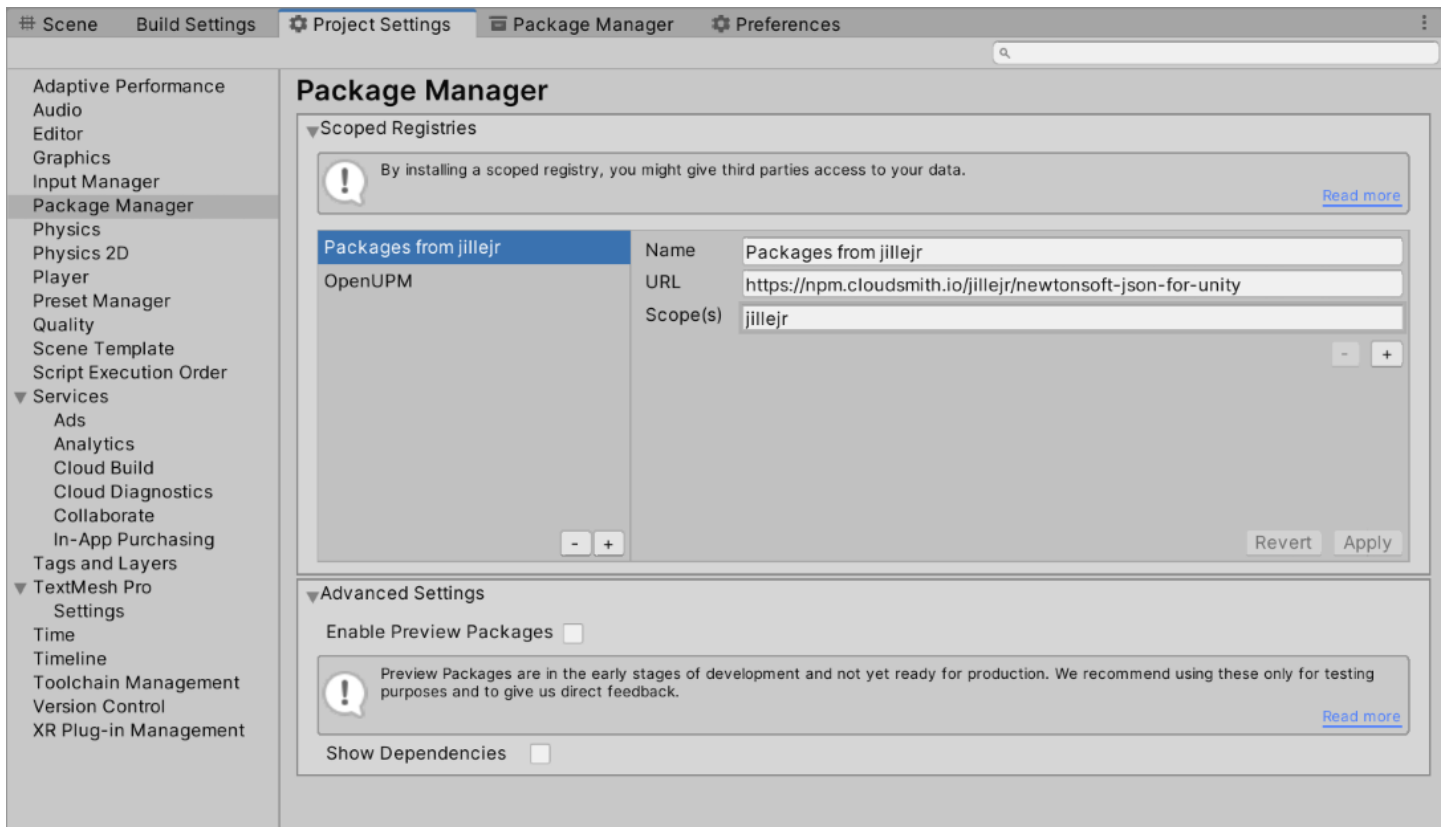
This step is optional. By adding these additional packages, you get to use more features *No Such Studio: Localization* as explained here.

These packages are hosted on 3rd party repositories. You need to add these repositories to Unity's UPM:

1. Open the *Project Settings* window
2. Go to the *Package Manager* tab
3. Add these two repositories.

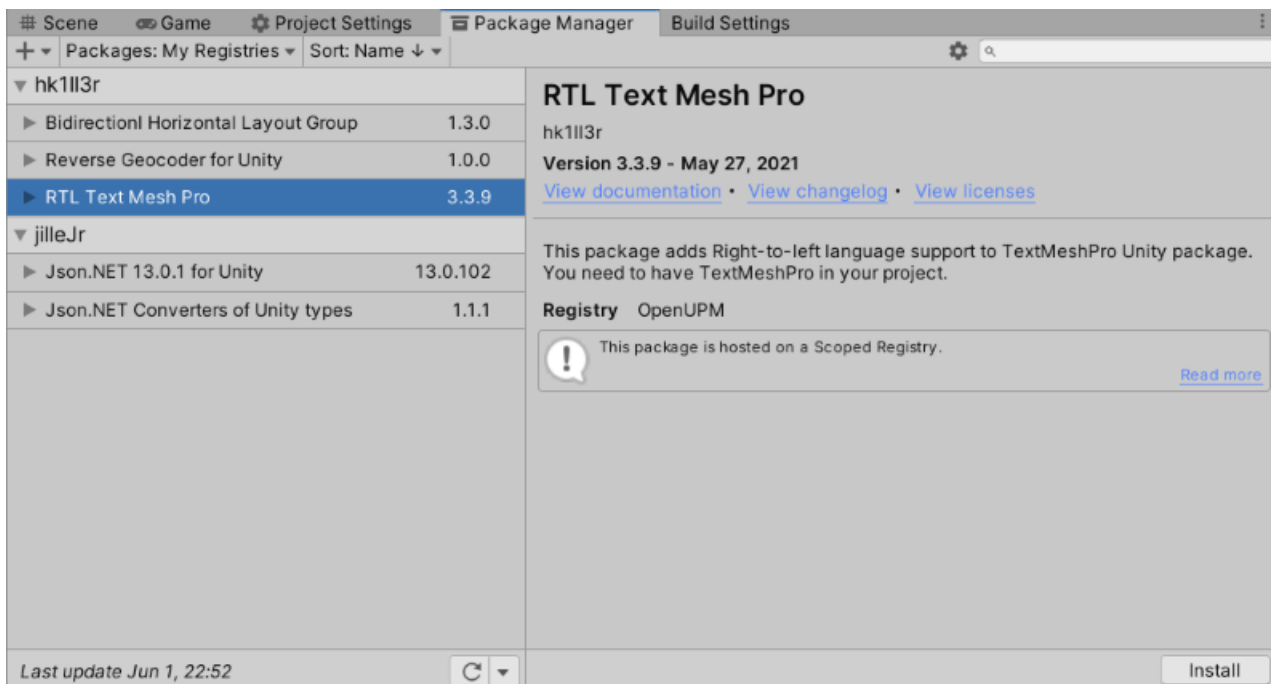
REPOSITORY NAME	URL	SCOPE
OpenUPM	https://package.openupm.com	com.nosuchstudio
Packages from jillejr	https://npm.cloudsmith.io/jillejr/newtonsoft-json-for-unity	jillejr





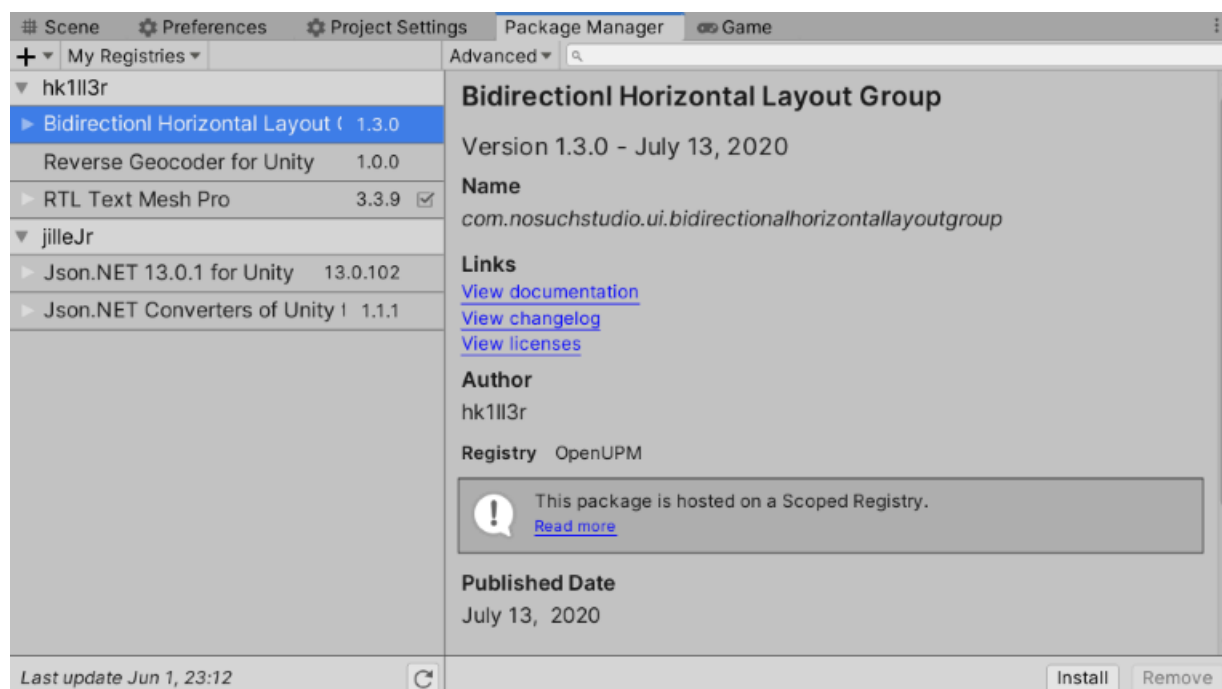
3.1 RTL Text Support

To use the RTL languages with Unity UI, add the following package.



3.2 RTL Layout Support for Unity 2019 and older

Unity 2020 and newer support both LTR and RTL layout with the `Horizontal Layout Group` component. For Unity 2019 and older, you can add this package to add RTL layout support.

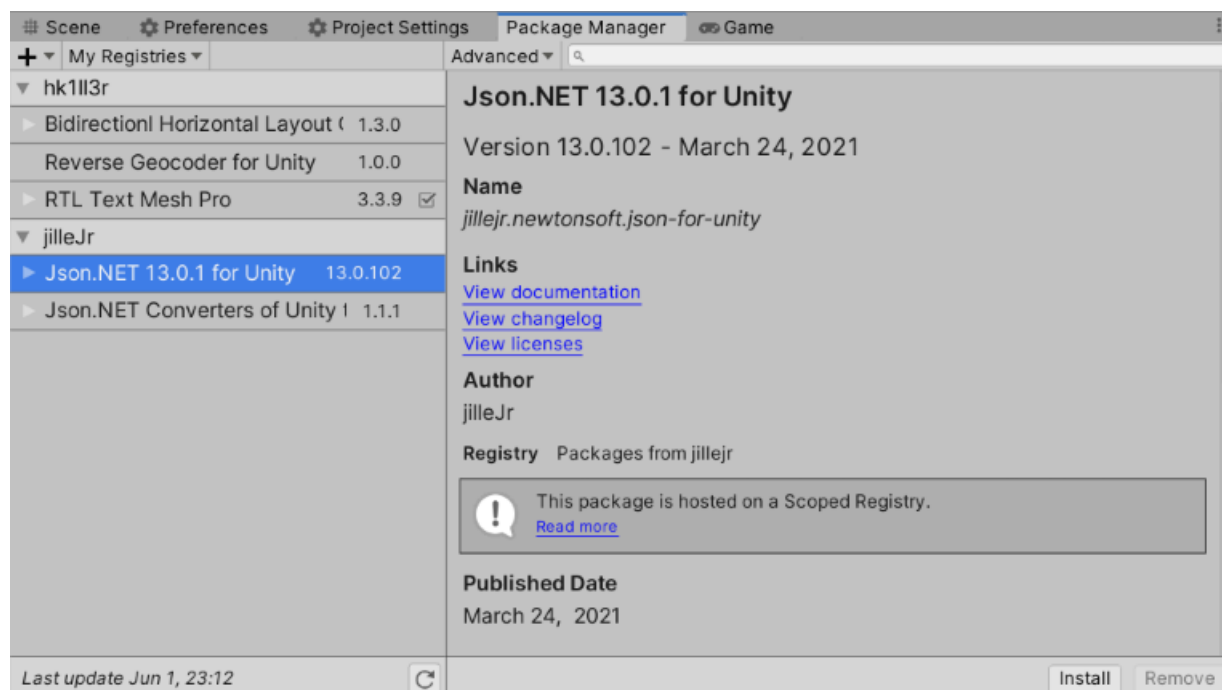


3.2 JSON Support for Unity 2019 and older

No Such Studio: Localization uses *NewtonSoft JSON* package when it is present. The following features depend on it:

- Typed variables
- Custom locales
- JSON translation source

In Unity 2020 and newer the package is included by default. In 2019 and older you can add the following package to use these features.



4. Explore the Demo Scenes

Add all the scenes to the *Build Settings* of your project.

Run the first scene named "0-SceneSelection". You can choose the demo scenes at runtime and by pressing the back button you'll go back to the scene selection page.

NOTE

The demo scenes depend on the additional packages from section 3. If you have not included all those packages, some scenes will not work properly.

Each scene has:

- A "Services" object which contains the singleton services.
- A "LocalizationSource" that contains all the translations for the scene.
- Buttons with country flags that change the language.
- Localizer Components that respond to language changes.

Take a few minutes to notice everything that changes when you switch the language, sprites, audio clips, layout direction and text.

Now exit the play mode and select the `Services` object in your scene.

- Notice that by toggling the `Localization Service` component, the service goes online and offline in all the localizer components in the scene.
- While the `Localization Service` is enabled, change the "Current Locale" field to another language. Notice that even in edit mode the language change takes effect.

See Also

- To get better understanding of the asset and how to use it, take a look at the [Overview](#) page.
- If you need to localize components that are not handled by the pre-made localizer components, you need to implement new localizer components. See [Extend No Such Localization](#) page to learn how.

Overview

Basics

`No Such Localization` consists of three categories of components.

1) `LocalizationService`

`LocalizationService` acts as the central hub for the localization framework. There must be a single instance of this component in the scene. I recommend marking the `GameObject` `DontDestroyOnLoad`. If there are multiple instances of this script, the one that gets enabled later will act as the registered and active instance.

Other components that participate in the localization process register with this service instance and are able to change the current language, listen to localization events and publish events.

2) `ComponentLocalizer<LT, CT>`

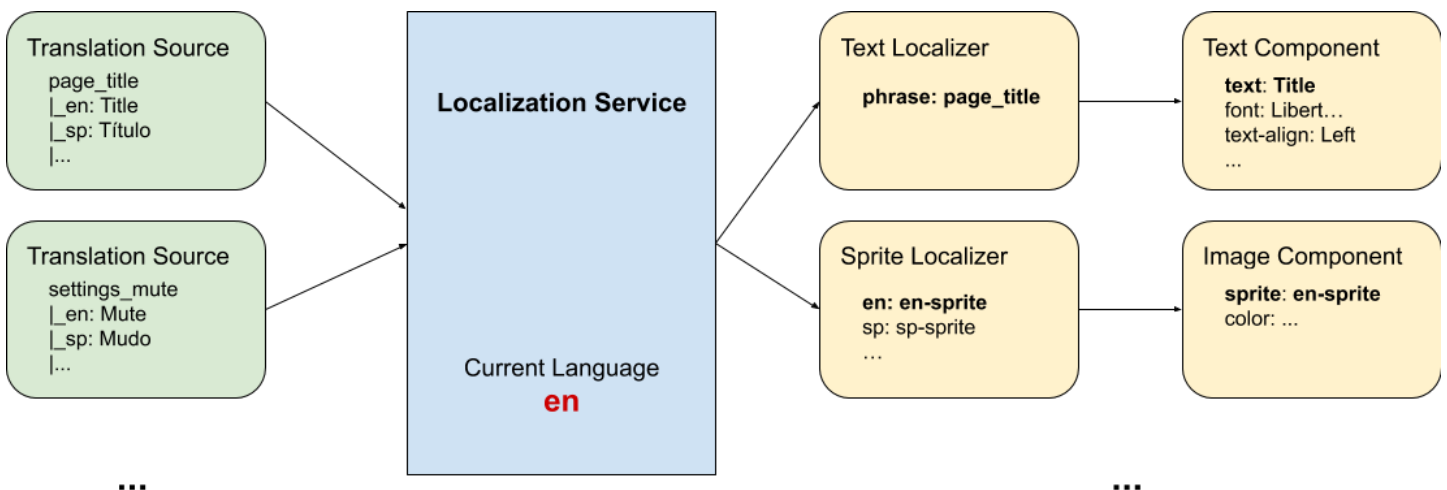
`ComponentLocalizer<LT, CT>` are components that make changes to other components based on the localization service's current language and translations and their own custom logic. The most prominent are Text localizers. The text of different UI components need to change and Text Localizers take care of that by registering to the `LocalizationService` and updating the text when current language changes.

Most `Localizer`s change only a single field of their target component. This way they can be mixed and matched to create more complex Localization behaviours. `SliderDirectionLocalizer` is a simple example of a `Localizer` component. It changes the direction of a `@Unity.UI.Slider` component when the language is right-to-left.

3) `BaseTranslationSource`

`Translation Source` components (subclassed from `BaseTranslationSource`) provide translations for phrases to the `LocalizationService`. In contrast to `Localizer` components, they don't listen nor react to the events of `LocalizationService` such as a language change.

This image shows how a menu UI is localized in a sample scene.



You can have as many translation sources and localizers as you need in your scene. The localizer components can find the right translation for a phrase through the `LocalizationService`. Since both translation sources and localizers are regular Unity components, you can save them in prefabs and reuse them in multiple scenes or load them dynamically.

You can organize translation sources depending on your needs:

- Distributed on individual `@UnityEngine.GameObject`s in your scene.
- Centralized in a single source in your scene
- One source per translated language

- etc.

It really depends on you, your game and your team. The architecture is flexible and gives you the options.

See Also

- Read the [Getting Started](#) page to see `No Such Localization` in action.
- Watch my [youtube tutorials](#).
- Read the [Extend No Such Localization](#) page to see how you can create your own custom components.

Extend No Such Localization

There are generally two ways to extend `No Such Localization`:

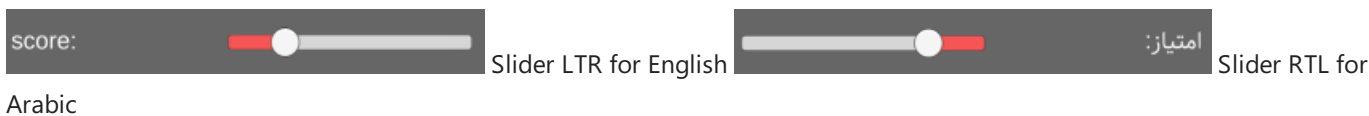
- Creating new `Localizer` components.
- Creating new `Translation Sources`.

Creating Localizer Components

The more common way of extending `No Such Localization` is writing new `Localizer` components. In this section we look at the code of three different `Localizer`s that are included in the Pro version of `No Such Localization`.

Creating a Basic Localizer: `SliderDirectionLocalizer`

This localizer toggles the `@UnityEngine.UI.Slider.direction` property of the Slider depending on the RTL-ness of current language.



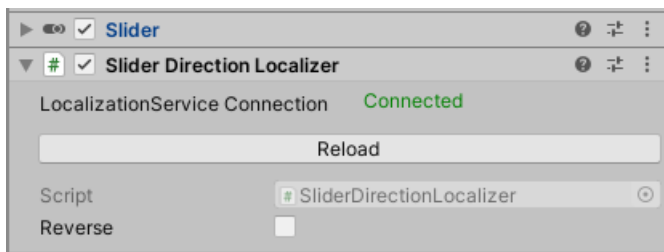
```
[RequireComponent(typeof(Slider))]  
public class SliderDirectionLocalizer : ComponentLocalizer<SliderDirectionLocalizer, Slider> {  
    public override void UpdateComponent() {  
        string lang = LocalizationService.CurrentLanguage;  
        _component.direction = LocalizationUtils.IsLangRTL(lang)  
            ? Slider.Direction.RightToLeft  
            : Slider.Direction.LeftToRight;  
    }  
}
```

The simplest way to create a new `Localizer` is to subclass `ComponentLocalizer<LT, CT>`. The type parameters are the class you are making and the component that your class localizes. Next you need to override the `UpdateComponent()` method. For this simple `Localizer` all that was needed was to set direction based on the RTL-ness of current language.

Editor

The default Editor often times suffices. Note that the it however lacks the common `Localization` service status. You can write an Editor to include the service status by extending the `ComponentLocalizerEditor` class.

```
[CustomEditor(typeof(SliderDirectionLocalizer))]  
public class SliderDirectionLocalizerEditor : ComponentLocalizerEditor<SliderDirectionLocalizerEditor, SliderDirectionLocalizer, Slider> {}
```



NOTE

If you are not familiar with Editor development in Unity, checkout [Unity's documentation website](#).

Creating a Text Localizer: `TextLocalizer`

This localizer changes the `text` property of a Text UI component. In addition to current language, the localizer component needs a

phrase such as "pageTitle" or "back_button_text" to translate into the current language. No Such Localization contains a base class for such localizers that need an additional phrase called `PhrasedComponentLocalizer<LT, CT>`.

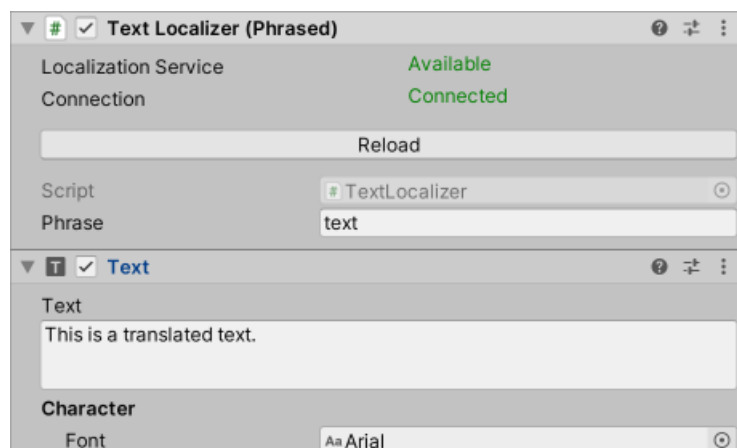
```
[RequireComponent(typeof(Text))]  
public class TextLocalizer : PhrasedComponentLocalizer<TextLocalizer, Text> {  
    public override void UpdatePhrasedComponent() {  
        _component.text = _translation;  
    }  
}
```

The only difference from the basic localizer discussed before is subclassing of `PhrasedComponentLocalizer<LT, CT>` and overriding `UpdatePhrasedComponent()`. `_translation` is the phrase translated to current language and you can use it when updating the target component.

Editor

The editor is going to be very similar to the basic localizer implemented in the previous section:

```
[CustomEditor(typeof(TextLocalizer))]  
public class TextLocalizerEditor : ComponentLocalizerEditor<TextLocalizerEditor, TextLocalizer, Text> {  
}
```



Creating an Asset Map Localizer: `ImageSpriteMappedLocalizer`

Another common type of `Localizer`s have a map from languages to an asset type (Sprite, Font, Alignment, etc.).

To create a `Localizer` of this category, you need to create two classes:

- AssetDataType class for the specific field you want to localize, i.e. Sprite, Font, etc. Note: Create these classes in `LocalizedAssetDataTypes.cs` file. You need these once per asset type. Declaring them in a central location avoids declaring them multiple time by mistake.

```
csharp [Serializable] public class LocalizedAssetDataSprite : LocalizedAssetData<Sprite> { public LocalizedAssetDataSprite(string name, Sprite data) : base(name, data) { } }
```

- Localizer class that subclasses `AssetMapComponentLocalizer<LT, CT, AT, LAD>`. `_assets` field contains the map of language -> asset that you create in Unity's Inspector. Use it to localize the target component.

```
csharp [RequireComponent(typeof(Image))] public class ImageSpriteMappedLocalizer :  
AssetMapComponentLocalizer<ImageSpriteMappedLocalizer, Image, Sprite, LocalizedAssetDataSprite> { public override void  
UpdateComponent() { string lang = LocalizationService.CurrentLanguage; if (string.IsNullOrEmpty(lang)) return; _component.sprite =  
_assets.ContainsKey(lang) && _assets[lang] != null ? _assets[lang] : _defaultAsset; } }
```

The editor is going to be similar to other editors discussed above.

▶

Image

🔍

↕

⋮

▼

Image Sprite Localizer (Mapped)

🔍

↕

⋮

Localization Service

Available

Connection

Connected

Reload

Script

ImageSpriteMappedLocalizer

Default Asset

us-globe

▼ Asset List

Size

6

▼ English

Name

English

Data

us-globe

▼ Spanish

Name

Spanish

Data

spain-globe

▼ Portuguese

Name

Portuguese

Data

brazil-globe

▶ Chinese

▶ Arabic

▶ Persian